

SYSTÈMES D'INFORMATION
& GÉNIE LOGICIEL — ARLA

Expense.io : audit et refacto d'architecture

Équipe 1Pacte

SYNTHÈSE EXÉCUTIVE

L’essentiel du projet

L’application marchait, mais elle exposait nos clients et coûtait cher à faire évoluer. Elle est maintenant sûre et prête pour la suite, sans rien changer pour nos clients.

1

Le risque métier

Une entreprise cliente pouvait lire les **justificatifs d’une autre** : perte de confiance, données personnelles exposées, contrats B2B en jeu. **4 failles critiques** sur 46 constats.

2

Ce qu’on a fait

Une **isolation des clients garantie** par la base, une architecture **en modules** et des e-mails fiables même si le serveur mail tombe. **Aucun client à réécrire** : l’API ne change pas.

3

Ce que ça apporte

Risque critique éliminé : 12 failles critiques et élevées corrigées, prouvées par 620 tests. Des évolutions **plus rapides et moins risquées** : une fonctionnalité complète en un prompt.

POUR L’ENTREPRISE Vendable sans craindre la fuite, évolutif sans tout casser.

1 · CE QU'ON A TROUVÉ

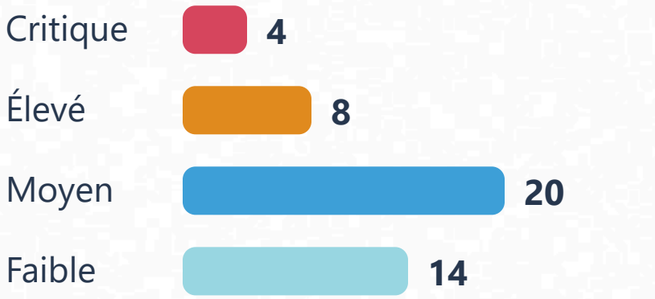
46 constats : l'isolation des clients d'abord

Classés par impact métier × facilité d'exploitation, tous rejoués dans l'app.

1 **Lire** l'existant

2 **Rejouer** l'app, rôle par rôle

3 **Figer** : +140 tests de caractérisation



8 élevés

Rôles usurpables, double décision acceptée, OCR qui sature le serveur, chemins serveur exposés.

Critique

Reçus d'une autre entreprise

Un manager ouvre le justificatif d'un salarié d'une autre entreprise.

Critique

Filet de la base désactivé

L'app se connecte en superuser : l'isolation par entreprise ne s'applique jamais.

Critique

Injection dans le tri des notes

`?sort=` exécute du SQL, avec les droits superuser.

Critique

Secret de session dans le code

Qui lit le dépôt peut fabriquer une session RH de n'importe quelle entreprise.

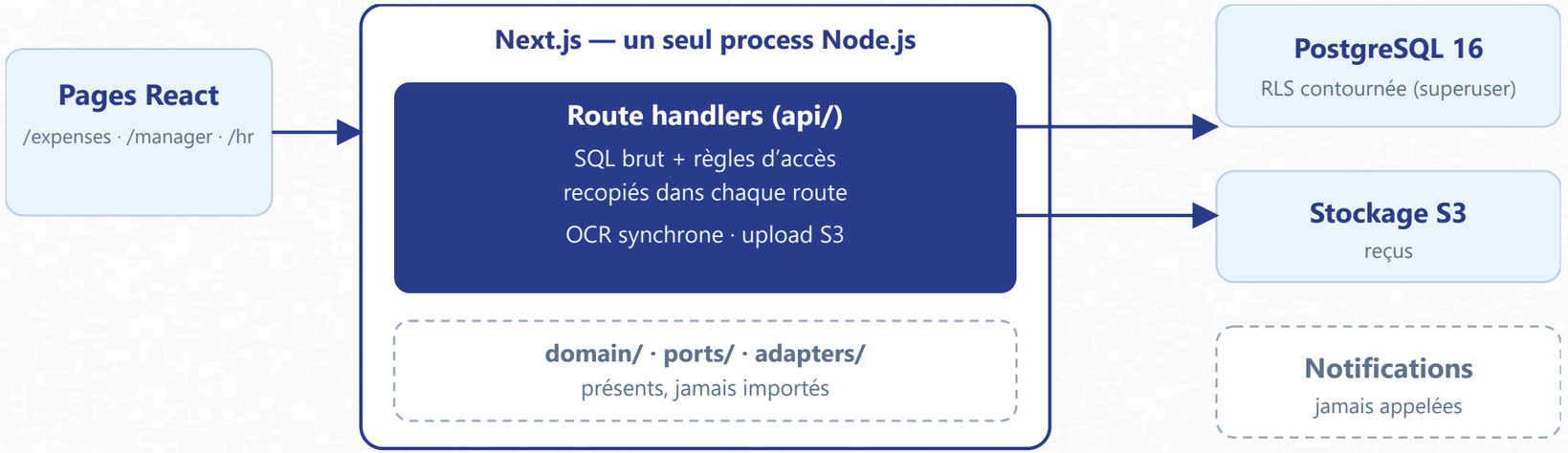
Et pour l'utilisateur : le salarié n'est **jamais prévenu** de la décision (e-mails codés, jamais envoyés), et la note d'un employé sans manager n'a **aucun validateur**.

POUR NOS CLIENTS Le justificatif d'un salarié pouvait être lu par une autre entreprise.

1 · CE QU'ON A TROUVÉ · ARCHITECTURE AS-IS

Un monolithe où tout se mélange

La couche hexagonale existait, mais personne ne l'importait.



7×
règles d'accès recopiées, une par route

0
filet dans la base : l'app s'y connecte en superuser

0
e-mail envoyé : les notifications ne sont jamais appelées

POUR NOS CLIENTS Une règle oubliée dans une route : un client voit les données d'un autre.

2 · CE QU'ON A DÉCIDÉ

Des frontières nettes, sans distribuer

Trois besoins tirés de l'audit : isoler les clients, ne rien casser chez eux, prévenir sans bloquer.

OPTION	APPORT	LIMITE
A · Monolithe en couches nettoyé	Léger.	Règles d'accès toujours éparpillées.
B · Monolithe modulaire hexagonal	Frontières nettes, métier testable.	Un serveur mail en panne bloque la validation.
C · B + file d'e-mails ✓	**Répond aux 3 besoins.**	**Un process de plus à exploiter (worker).**
D · Microservices	Déploiement indépendant.	Réseau et exploitation ×N, pour aucun besoin de charge.

Isoler : la base l'impose

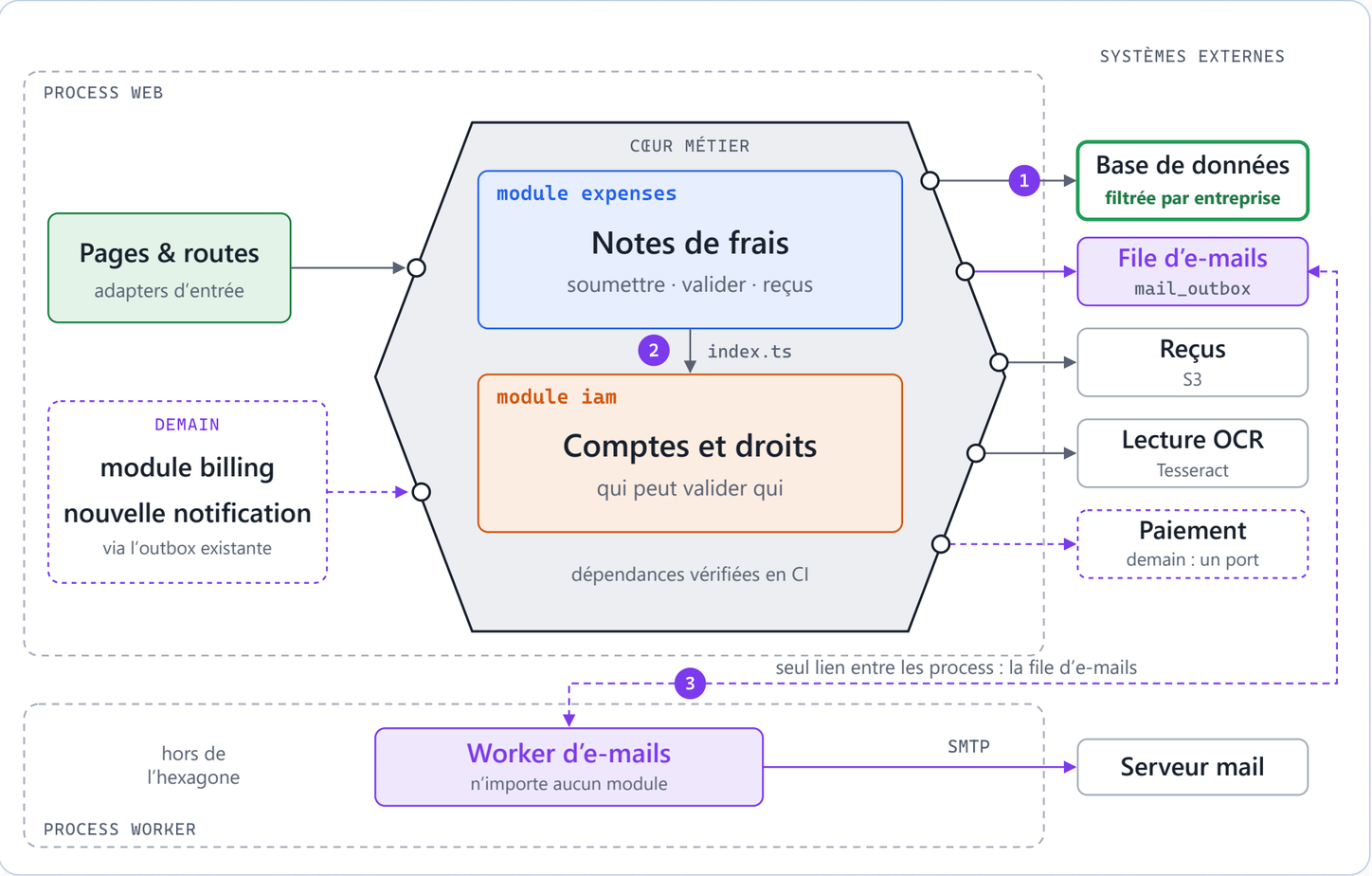
Ne rien casser : API figée

Charge : 360–560 req/s suffisent

Choix écrit : ADR 0002

POUR L'ENTREPRISE Une seule application à exploiter, découpée pour accueillir la suite.

Isolée par la base, découpée, fiable



1. **Isoler les clients.** Chaque requête passe par la base, qui ne rend que les données de l'entreprise de l'utilisateur. Un oubli dans le code ne fuit plus.
2. **Évoluer sans casser.** Un module n'expose qu'une porte ; la CI refuse tout import interdit. Le nouveau se branche à côté.
3. **Prévenir sans bloquer.** Décision et e-mail sont enregistrés ensemble ; un worker séparé envoie. Serveur mail en panne : la validation passe, l'e-mail part plus tard.

POUR NOS CLIENTS Leurs données restent chez eux, et le salarié est toujours prévenu.

3 · LA PREUVE

Les 4 failles critiques fermées

Chaque correction est tenue par un test qui échouerait si le défaut revenait.

FAILLE	CORRECTION	PREUVE (TEST)
Reçus d'une autre entreprise	Accès vérifié par le module, puis par la base	chaque rôle, autre entreprise → 404
Filet de la base désactivé	Rôle applicatif soumis à l'isolation	sans entreprise, rien n'est visible
Injection dans le tri	Liste blanche des colonnes	`?sort=` inconnu → tri par défaut
Secret dans le code	Secret obligatoire à l'environnement	sans secret, l'app refuse de démarrer

MOYENS ET FAIBLES **17 corrigés sur 34** (11 moyens, 6 faibles). Les 15 écartés le sont pour une raison écrite : contrat, simple validation, fonctionnalité ou infrastructure.

LIMITE ASSUMÉE L'OCR reste synchrone : le rendre asynchrone changerait l'API. Risque atténué (taille et concurrence plafonnées, annexe 16).

POUR NOS CLIENTS Mêmes parcours, même API, et des données enfin isolées entre entreprises.

12/12

critiques et élevés corrigés · 29 défauts au total

10 → 0

parcours E2E en échec : l'app d'origine en ratait 10 sur 37, la nôtre passe **37/37**

99,8 %

des lignes du cœur métier et de l'API, par **620 tests**. Sonar, sur tout `src/` : **75 %**, l'écart étant l'interface testée en E2E.

4 · LA DÉMONSTRATION

Une fonctionnalité, un prompt

Chaque outil ferme un risque précis : l'agent intègre, il n'improvise pas.

1

AGENTS.md

Architecture, règles et commandes, chargés à chaque session.

Évite : du code posé au mauvais endroit

2

Skills

Recette TDD en 10 étapes : `ajouter-une-fonctionnalite`.

Évite : un test, le contrat d'API ou la doc oubliés

3

Revue

Sous-agent `revue-architecture` sur le diff.

Évite : une règle contournée qui compile quand même

4

Garde-fous

`npm run verify` : lint, types, depcruise, 620 tests.

Évite : un import interdit ou une régression

Lisible par l'humain aussi | ADR : pourquoi chaque choix | Conventions : où va chaque fichier | Runbooks et CHANGELOG du contrat

Déjà testé	≈ 12 min jusqu'au vert	0 intervention humaine	0 constat bloquant

MERCI Questions, puis live coding.

ANNEXES

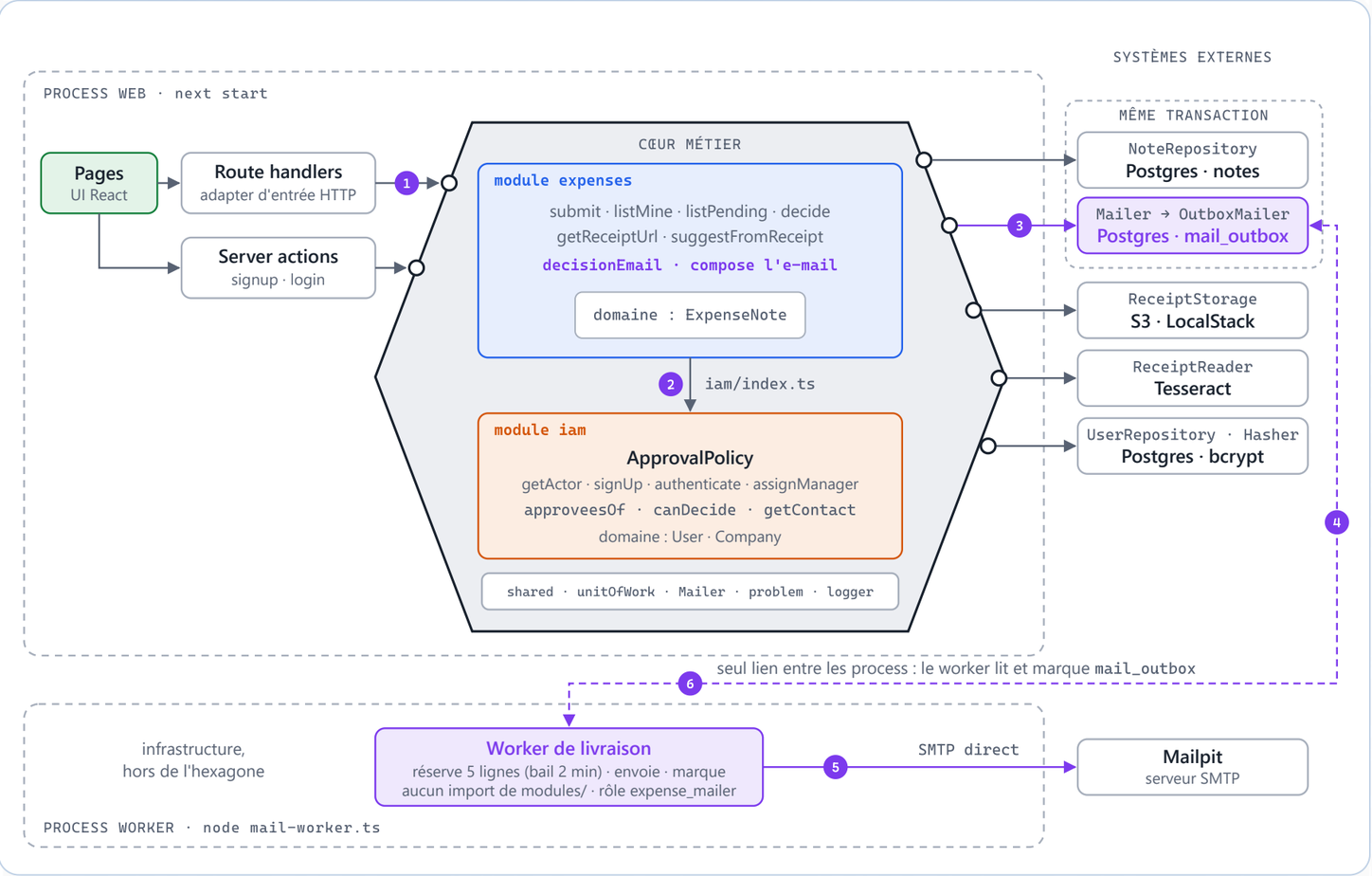
Pour aller plus loin



Architecture détaillée, méthode d’audit, gravité, couverture, moyens et faibles, ADR 0002, OCR.

ANNEXE · ARCHITECTURE DÉTAILLÉE

Monolithe modulaire, hexagonal, avec outbox



1. Le manager valide une note : **l'API répond comme avant**.
2. « Qui peut valider » est écrit **à un seul endroit** ; deux validations simultanées, une seule gagne.
3. La décision et l'e-mail sont enregistrés **ensemble** : jamais l'un sans l'autre.
4. Un process séparé prend les e-mails en attente ; **s'il plante, il reprend** où il en était.
5. Une panne du serveur mail **ne bloque jamais** une validation.
6. Échec : nouvel essai, puis mise de côté. **Un doublon est possible, une perte jamais.**

À RETENIR Règles vérifiées en CI ; les deux process ne se parlent que via `mail_outbox` .

ANNEXE · MÉTHODE D'AUDIT

Cinq étapes, chaque constat prouvé

Avant de toucher au code : comprendre, rejouer, mesurer, figer, puis classer.

1

Cartographier

Lecture des routes, de l'authentification et des migrations. Schéma as-is.

2

Rejouer

App lancée en local, parcourue dans un vrai navigateur pour chaque rôle ; API rejouée avec le cookie réel de chaque rôle.

3

Mesurer

k6 sur 15 000 notes (169 147 requêtes) ; SonarQube et couverture de départ.

4

Figer

+140 tests de caractérisation sur Postgres et S3 réels. Les tests `[as-is]` ont révélé 8 constats de plus.

5

Classer

Gravité = impact métier × facilité d'exploitation (annexe 12).

19 confirmés : reproduits sur l'app

20 établis par le code : lecture

7 par un test : suite de caractérisation

À RETENIR Chaque constat a un niveau de preuve, les 4 critiques compris.

ANNEXE · GRILLE DE GRAVITÉ

Critique ou élevé : la condition d'exploitation

Même impact possible ; ce qui les sépare, c'est ce qu'il faut pour l'exploiter.

Critique · 4

Sans condition

N'importe quel utilisateur, en une requête, sur toutes les entreprises ; ou la défense elle-même désactivée.

- **F1** : tout manager lit les reçus de toute entreprise
- **F2** : la base ne protège plus rien
- **F3** : un paramètre exécute du SQL superuser
- **F25** : avec le dépôt, toute session se fabrique

Élevé · 8

Grave, mais sous condition

Il faut un rôle, un moment ou une charge particulière, ou l'impact reste indirect.

- **F4, F5, F30** : être le premier inscrit, ou RH qui agit
- **F9** : deux décisions au même instant
- **F16, F17, F19** : saturer le serveur (charge, gros fichier)
- **F26** : une stack exposée, qui aide une attaque

Moyen : impact métier net, sans fuite (ex. salarié jamais prévenu, note sans validateur). **Faible** : correction souhaitable (affichage, validations fines).

À RETENIR F1 et F5 exposent une autre entreprise ; seul F1 n'exige rien.

ANNEXE · COUVERTURE DES TESTS

99,8 % mesuré, 75 % dans Sonar

Même suite de tests, deux périmètres : l'écart vient de ce que Vitest ne mesure pas.

ZONE	TESTÉE PAR	COUVERTURE
Cœur métier, API, server actions modules/, shared/, app/api/, app/actions/	Vitest : 620 tests, Postgres et S3 réels	**99,8 %** des lignes · 97,4 % des branches · 100 % des fonctions
Pages et composants React 7 pages, 4 composants	Playwright : 37 parcours E2E	non mesurée, comptée à 0 % par Sonar
Authentification lib/auth.ts, proxy.ts, route Auth.js	Playwright : vraies sessions	non mesurée
Worker d'e-mails worker/mail-worker.ts	Vitest (intégration) et E2E Mailpit	testé, hors du périmètre mesuré

À RETENIR On n'a pas exclu l'interface de Sonar pour gonfler le chiffre : on montre les deux.

ANNEXE · MOYENS ET FAIBLES

Moyens et faibles : 17 corrigés sur 34

Rien n’est oublié : chaque constat est corrigé ou écarté pour une raison écrite.

GRAVITÉ	CORRIGÉS	ÉCARTÉS, ET POURQUOI
Moyen · 20	**11** corrigés, dont e-mails (F15), notes sans validateur (F14, F33) + 1 déjà traité (F35), 1 partiel (F21)	**7** : pagination = contrat (F18) ; validations (F13, F32, F40) ; débit = infrastructure (F20) ; identité d’entreprise à concevoir (F6, F7)
Faible · 14	**6** corrigés	**8** : fonctionnalités manquantes (F8, F28, F38) ; validations (F37, F45) ; affichage (F29, F46) ; énumération des comptes (F36)

Contrat figé : rien qui change l’API

Fonctionnalités : candidates au live

Aucun critique ni élevé écarté

À RETENIR Écarter, c’est décider : chaque exclusion est écrite dans la spec, avec sa raison.

ANNEXE · ADR 0002

ADR 0002 : monolithe modulaire hexagonal

Acceptée le 16/10/2026 · docs/adrs/0002-monolithe-modulaire-hexagonal.md

CONTEXTE

Routes qui portent le SQL et les règles d'accès, recopiées 7 fois ; couche hexagonale présente mais jamais branchée. Contrainte : ne pas casser l'API.

OPTIONS

A couches nettoyées · **B** modulaire hexagonal · **C** B + flux asynchrone · **D** microservices

DÉCISION : C

Modules `expenses` et `iam` (domain, application, ports, infrastructure) et `shared/`. Un module ne voit l'autre que par son `index.ts`, vérifié par dependency-cruiser. **ApprovalPolicy** seule décide qui valide ; la RLS n'assure que l'isolation entre entreprises.

CONSÉQUENCES +

Une règle métier écrite une fois ; une recette fixe pour ajouter une fonctionnalité.

CONSÉQUENCES –

Plus de fichiers. Une RH seule dans son entreprise n'a pas de validateur tant qu'une seconde RH n'existe pas (limite assumée).

SUIVI

La facturation ira dans `src/modules/billing`, avec un port `PaymentProvider`.

À RETENIR Le choix est écrit, daté, avec ses coûts : n'importe qui (humain ou agent) peut le relire.

ANNEXE · OCR SYNCHRONE

L'OCR : borné plutôt que déplacé

Le risque était la saturation (F17, F19), pas la lenteur : plafonnée sans toucher à l'API.

POURQUOI SYNCHRONE

La suggestion renvoie montant et date **dans sa réponse**. En asynchrone : 202 puis une route de résultat, donc **les clients existants cassent**.

CE QUE COÛTE L'OCR

≈ 450 ms par reçu (k6), acceptable pour une saisie humaine.

DEMAIN, SI LA CHARGE L'EXIGE

`ReceiptReader` est un port : un adapter avec file d'attente remplace Tesseract sans toucher au métier.

AVANT	APRÈS
Un moteur Tesseract créé puis détruit **à chaque reçu** (F17)	Moteurs **créés une fois, réutilisés**
Autant d'analyses que de requêtes simultanées	**2 analyses au plus** en parallèle (`OCR_MAX_CONCURRENCY`), les autres attendent leur tour, 30 s au plus
Saturé : le serveur ralentit pour tout le monde	**503 + Retry-After : 5 s**, « réessaie dans quelques secondes » ; ajouté au contrat
Aucune limite de taille (F19)	**10 Mio**, corps coupé dès le dépassement, **400** sans analyse
OCR en échec	**Suggestion vide** : le salarié saisit à la main, rien ne bloque

À RETENIR Borné aujourd'hui, déplaçable demain, sans casser l'API.